

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

An Autonomous Institute | Affiliated to Osmania University

Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in

Approved by | Affiliated to | UGC Autonomous | 14 Programs Accredited by NBA | Grade A++ in NAAC | All India Ranking 151-200 Band nirf | Certifications: Quality ISO 21001 : 2018, Green ISO 14001 : 2015, Energy ISO 50001 : 2018

COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

AI/ML-Based Identification, Classification, and Prediction of Tropical Cyclone Patterns Using Multi-Source Satellite Data

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Osmania University)

CASE STUDY REPORT

Submitted By:

160124748087- R SIRIVALLI

160124748090- S SAI SAHASRA REDDY

CASE STUDY GUIDE

Panigrahi Srikanth
Assistant Professor
Department of CSE AI&ML
CBIT, Hyderabad

Department of CSE Artificial Intelligence and Machine Learning
CBIT, Hyderabad

Certificate

This is to certify that the case study report entitled **AI/ML-Based Identification, Classification, and Prediction of Tropical Cyclone Patterns Using Multi-Source Satellite Data** has been carried out by the students listed below during the academic year 2026–2027 under the guidance and supervision of the faculty guide.

160124748087- R SIRIVALLI

160124748090- S SAI SAHASRA REDDY

Project Guide:

Panigrahi Srikanth
Assistant Professor
Department of CSE AI&ML
CBIT, Hyderabad

Acknowledgement

We express our sincere gratitude to our case study guide, **Panigrahi Srikanth**, for his guidance, valuable suggestions, and encouragement throughout this work. We thank the Head of the Department and the faculty members of the Department of CSE Artificial Intelligence and Machine Learning, Chaitanya Bharathi Institute of Technology, for providing the academic environment and support required to complete this case study. We also acknowledge NOAA for the satellite dataset and the developers of the open-source tools used in the implementation. Finally, we thank our families and friends for their continued support and motivation.

Abstract

Tropical cyclones are intense weather systems that can cause severe damage to coastal communities, infrastructure, and ecosystems. Timely identification and intensity classification of cyclones are important for monitoring and disaster preparedness. This case study presents a deep-learning pipeline for identifying and classifying tropical cyclone patterns from multi-channel satellite observations. The implementation uses NOAA HURSAT-B1 data and selected channels, including IRWIN, IRWVP, and VSCHN, where available in the notebook data. The proposed MLFF-CycloneNet architecture combines convolutional feature extraction, multi-level feature fusion, and channel and spatial attention for classifying samples into Tropical Depression, Tropical Storm, and Hurricane/Typhoon categories. A separate detector is used for cyclone versus background classification. The implementation is developed in Python using PyTorch and supporting scientific-computing libraries. Evaluation includes standard classification metrics, confusion matrices, training curves, and Grad-CAM visualizations. This report describes the theoretical background, data pipeline, model design, experimental workflow, results to be filled from the executed notebook, limitations, and future improvements. The present implementation performs image-based detection and intensity classification; it should not be interpreted as a validated future-track forecasting system.

List of Figures

4.1 Overall architecture of the tropical cyclone analysis system 11

List of Tables

3.1 Hardware and computational requirements 6
3.2 Software and frameworks used 6
3.3 Dataset characteristics 7
3.4 Wind-speed-based intensity categories 7

6.1 MLFF-CycloneNet test performance 15
6.2 MLFFCycloneDetector test performance 15

8.1 Hyperparameters to verify against the executed notebook 20

Contents

Certificate i

Acknowledgement ii

Abstract iii

List of Figures iv

List of Tables v

Table of Contents vi

1 Introduction 1

1.1 Background and Significance 1
1.2 Deep Learning for Tropical Cyclone Analysis 1
1.3 Problem Definition 1
1.4 Challenges in Tropical Cyclone Identification 2
1.4.1 Variation in Cyclone Structure 2
1.4.2 Multi-Channel Data 2
1.4.3 Class Imbalance 2
1.4.4 Temporal and Storm-Level Correlation 2
1.4.5 Reliable Background Samples 2
1.5 Objectives 2
1.6 Motivation and Report Structure 3

2 Literature Review and Theoretical Study 4

2.1 Evolution of Satellite Image Analysis 4
2.2 Convolutional Neural Networks 4
2.3 Transferable and Hierarchical Features 4
2.4 Attention Mechanisms 4
2.4.1 Channel Attention 4
2.4.2 Spatial Attention 4
2.4.3 Attention in Satellite Analysis 5
2.5 Grad-CAM for Model Interpretation 5
2.6 Evaluation of Classification Models 5
2.7 Research Gap 5

3 System Requirements and Specifications 6
3.1 Hardware and Software Requirements 6
3.1.1 Hardware Specifications 6
3.1.2 Software and Frameworks 6
3.2 Dataset Description and Characteristics 6
3.3 Intensity Labels 7
3.4 Functional Requirements 7
3.5 Non-Functional Requirements 8

4 Proposed System and Methodology 9
4.1 Overall Workflow 9
4.2 Data Acquisition and Inspection 9
4.3 Preprocessing Pipeline 9
4.3.1 Channel Selection 9
4.3.2 Array Construction 9
4.3.3 Normalization 9
4.3.4 Label Generation 9
4.4 Dataset Splitting 10
4.5 Proposed MLFF-CycloneNet 10
4.5.1 Convolutional Feature Extraction 10
4.5.2 Multi-Level Feature Fusion 10
4.5.3 Channel and Spatial Attention 10
4.5.4 Classification Head 10
4.6 Cyclone Detection Model 10
4.7 System Architecture 11
4.8 Loss Function 11
4.9 Optimizer and Learning-Rate Strategy 11

5 Implementation and Experimental Setup 12
5.1 Implementation Environment 12
5.2 Data Loading and Dataset Class 12
5.3 Model Training Strategy 12
5.3.1 Training and Validation 12
5.3.2 Batch Processing 12
5.3.3 Model Checkpointing 12
5.3.4 Learning-Rate Scheduling 12
5.4 Experimental Procedure 13
5.5 Prediction Pipeline 13
5.6 Saved Artifacts 13

6 Results and Performance Analysis 14
6.1 Overview 14
6.2 Quantitative Performance Metrics 14

6.2.1 Accuracy 14
6.2.2 Precision 14
6.2.3 Recall 14
6.2.4 F1-Score 14
6.3 Intensity Classification Results 15
6.4 Binary Detection Results 15
6.5 Training and Validation Analysis 15
6.6 Confusion Matrix Analysis 15
6.7 Precision, Recall, and F1-Score Analysis 16
6.8 Grad-CAM and Qualitative Analysis 16
6.9 Robustness and Generalization 16
6.10 Summary of Findings 16

7 Conclusion and Future Scope 17
7.1 Conclusion 17
7.2 Limitations 17
7.3 Future Scope 17
7.3.1 Storm-Wise Evaluation 17
7.3.2 Verified Background Dataset 17
7.3.3 Additional Satellite Channels 18
7.3.4 Temporal Forecasting 18
7.3.5 Uncertainty Estimation 18
7.3.6 Operational Visualization 18

8 Appendix I: Model Details and Preprocessing 19
8.1 MLFF-CycloneNet Model Details 19
8.2 Model Training Strategy 19
8.3 Preprocessing Pipeline 19
8.4 Data Augmentation 19
8.5 Classification Head Details 20
8.6 Important Hyperparameters 20

9 References 21
9.1 Reference Categories 21
9.2 Bibliographic References 21
9.3 Software and Online Resources 21
9.4 Reference Summary 21

Introduction

1.1 Background and Significance

Tropical cyclones are rotating low-pressure systems that form over warm tropical oceans. They are associated with strong winds, heavy rainfall, storm surges, and flooding. The impact of a cyclone depends on its intensity, movement, structure, and the vulnerability of affected regions. Accurate and timely observation is therefore important for weather monitoring and disaster-risk reduction.

Satellite remote sensing provides repeated observations over large oceanic areas where conventional observations may be limited. Infrared and water-vapour channels reveal different aspects of cloud-top temperature, moisture, and atmospheric structure. Deep learning can learn spatial representations from these observations and assist in identifying cyclone-related patterns.

1.2 Deep Learning for Tropical Cyclone Analysis

Traditional analysis depends on meteorological interpretation, handcrafted indicators, and expert assessment. Deep learning methods such as convolutional neural networks (CNNs) learn image features directly from training data. Attention mechanisms can help emphasize informative feature channels or spatial regions, while feature fusion combines information extracted at different network depths.

1.3 Problem Definition

The problem addressed in this case study is to process multi-channel satellite observations and predict cyclone-related labels. For a satellite input X, a classification model estimates a class probability distribution:

$$P(y = k \mid X), \qquad k \in \{0, 1, \ldots, K-1\}.$$

The predicted class is:

$$\hat{y} = \arg\max_{k} P(y = k \mid X).$$

The implemented intensity labels are Tropical Depression, Tropical Storm, and Hurricane/Typhoon. A separate binary component classifies input patches as cyclone or background. These outputs represent image-based classification, not a forecast of a future cyclone track.

1.4 Challenges in Tropical Cyclone Identification

1.4.1 Variation in Cyclone Structure

Cyclones differ in size, organization, cloud-band shape, and eye formation. Their appearance also changes as they develop or weaken.

1.4.2 Multi-Channel Data

Different satellite channels have different physical interpretations, scales, and distributions. The input pipeline must handle channel selection and normalization consistently.

1.4.3 Class Imbalance

The number of samples in each intensity category may differ. Imbalance can bias the model toward classes with more examples and should be assessed using class-wise metrics.

1.4.4 Temporal and Storm-Level Correlation

Observations from the same storm are often correlated. If images from a storm appear in both training and testing sets, the measured test performance may overestimate generalization to unseen storms.

1.4.5 Reliable Background Samples

Background patches extracted from a cyclone-centred image may still contain cyclone-related structures. Independent, verified non-cyclone samples are needed for a reliable operational detector.

1.5 Objectives

1. Prepare multi-channel satellite observations for deep-learning input.
2. Generate cyclone intensity labels from the wind-speed metadata available in the dataset.
3. Implement an MLFF-CycloneNet classifier using convolutional feature extraction and attention.
4. Implement a separate cyclone-versus-background detector.
5. Evaluate predictions using accuracy, precision, recall, F1-score, and confusion matrices.

6. Visualize training behaviour and selected predictions using training curves and Grad-CAM.
7. Document limitations and identify improvements for storm-independent evaluation and forecasting.

1.6 Motivation and Report Structure

The motivation is to study how deep learning can support satellite-image analysis for tropical cyclone monitoring. Chapter 2 reviews theoretical foundations and related methods. Chapter 3 describes the system requirements and dataset. Chapter 4 presents the proposed system and methodology. Chapter 5 explains the implementation and experimental procedure. Chapter 6 discusses evaluation and results. Chapter 7 presents the conclusion and future scope. Chapter 8 provides model and preprocessing details, and Chapter 9 lists references.

Literature Review and Theoretical Study

2.1 Evolution of Satellite Image Analysis

Early image-analysis systems relied on manually selected image features and rules. CNNs made it possible to learn hierarchical spatial representations directly from data. Current approaches increasingly combine pretrained networks, attention, multi-scale feature extraction, and interpretability methods.

2.2 Convolutional Neural Networks

A CNN applies learnable filters to local regions of an input. Convolutional layers detect patterns, activation functions introduce non-linearity, and pooling or strided operations reduce spatial dimensions. For satellite imagery, these layers can represent cloud edges, bands, textures, and larger structural patterns.

2.3 Transferable and Hierarchical Features

Features in shallow layers tend to preserve local edges and textures, whereas deeper layers capture more abstract patterns. Combining information from multiple levels can retain local detail while providing high-level context. The case-study architecture uses multi-level feature fusion to combine such representations.

2.4 Attention Mechanisms

2.4.1 Channel Attention

Channel attention estimates the importance of feature channels and reweights them to emphasize useful information.

2.4.2 Spatial Attention

Spatial attention assigns weights to different spatial locations in a feature map, allowing the network to focus on potentially relevant regions.

2.4.3 Attention in Satellite Analysis

Attention may improve feature selection, but its effectiveness must be established through empirical comparison and should not be assumed from the architecture alone.

2.5 Grad-CAM for Model Interpretation

Gradient-weighted Class Activation Mapping (Grad-CAM) uses gradients flowing into a convolutional feature map to produce a heatmap associated with a predicted class. It can help inspect which regions influenced a prediction. It is a qualitative explanation and does not establish meteorological correctness or causality.

2.6 Evaluation of Classification Models

Accuracy gives the overall proportion of correct predictions. Precision measures how many predicted positives are correct, recall measures how many actual positives are identified, and the F1-score balances precision and recall. Confusion matrices show the distribution of correct and incorrect class predictions. For imbalanced data, class-wise and macro-averaged metrics should be examined in addition to accuracy.

2.7 Research Gap

Satellite-image classification must handle different cyclone structures, imbalanced classes, and correlated observations. A random sample-level split may not fully measure generalization to unseen storms. The case study brings together multi-channel preprocessing, feature fusion, attention, a separate detector, and visualization. Further validation using independent storms and verified non-cyclone samples is needed before practical use.

System Requirements and Specifications

3.1 Hardware and Software Requirements

3.1.1 Hardware Specifications

Table 3.1: Hardware and computational requirements

Component	Requirement / Description
Development platform	Google Colaboratory
Processor	Cloud-provided CPU
GPU	GPU accelerator when available in the Colab runtime
Memory	Runtime-dependent RAM
Storage	Temporary Colab storage or connected cloud storage
Input type	Multi-channel satellite observations
Output	Cyclone detection label and intensity class

3.1.2 Software and Frameworks

Table 3.2: Software and frameworks used

Software / Library	Purpose
Python	Main programming language
Google Colab	Cloud notebook environment
PyTorch	Neural-network definition and training
Torchvision	Image and tensor utilities where required
NumPy and Pandas	Numerical operations and metadata processing
xarray, netCDF4, h5netcdf	Reading NetCDF satellite data
scikit-learn	Dataset splitting and evaluation metrics
Matplotlib and Seaborn	Training plots and confusion matrices
tqdm and Requests	Progress display and data retrieval
pytorch-grad-cam	Grad-CAM visualizations

3.2 Dataset Description and Characteristics

The notebook uses NOAA HURSAT-B1 satellite data. The data are stored in NetCDF-based files and may contain channel variables and storm metadata. The implementation selects chan-

nels named `IRWIN`, `IRWVP`, and `VSCHN`, depending on availability in the source files. Metadata such as storm name, observation time, wind speed, and central pressure may be used when available.

Table 3.3: Dataset characteristics

Property	**Description**
Dataset	NOAA HURSAT-B1
Data type	Satellite observations stored in NetCDF files
Selected channels	IRWIN, IRWVP, VSCHN, subject to source availability
Primary task	Three-class cyclone intensity classification
Secondary task	Binary cyclone/background classification
Metadata	Storm information and wind speed when available
Input representation	Three-channel numerical array
Sample count	Insert count printed by the executed notebook

3.3 Intensity Labels

The implementation maps wind speed in knots into three intensity classes according to the thresholds used in the notebook.

Table 3.4: Wind-speed-based intensity categories

Label	**Class**	**Rule**
0	Tropical Depression	Wind speed below 34 knots
1	Tropical Storm	Wind speed from 34 to below 64 knots
2	Hurricane/Typhoon	Wind speed of 64 knots or greater

3.4 Functional Requirements

- Load satellite data and inspect required variables.
- Prepare channel arrays and associated labels.
- Normalize inputs and create training, validation, and test partitions.
- Train the detection and intensity-classification models.
- Save checkpoints and training history.
- Produce evaluation reports, confusion matrices, and interpretability plots.
- Predict labels for supplied samples.

3.5 Non-Functional Requirements

The system should provide reproducible preprocessing, clear evaluation output, manageable training time, and consistent model saving. The implementation should avoid data leakage between splits and report limitations honestly.

Proposed System and Methodology

4.1 Overall Workflow

The workflow consists of data retrieval, extraction, NetCDF inspection, metadata processing, class labelling, channel-array preparation, normalization, data splitting, model training, evaluation, visualization, and sample prediction.

4.2 Data Acquisition and Inspection

The notebook obtains HURSAT-B1 files and inspects their variables and metadata. The data loader identifies the selected satellite channels and relevant metadata fields. Missing variables and invalid samples must be handled before model training.

4.3 Preprocessing Pipeline

4.3.1 Channel Selection

The input is assembled from the selected channels `IRWIN`, `IRWVP`, and `VSCHN`. Each channel contributes a different view of the atmospheric scene.

4.3.2 Array Construction

The selected channels are combined into a consistent three-channel array. Shapes and data types are checked so that samples can be batched by the PyTorch data pipeline.

4.3.3 Normalization

Channel values are standardized using the mean and standard deviation used by the notebook. A small threshold is applied to avoid division by a near-zero standard deviation. The same preprocessing must be applied during evaluation and prediction.

4.3.4 Label Generation

Wind-speed metadata are mapped to the three intensity categories. Samples without valid metadata or required channel data must be excluded or handled according to the notebook's implemented rules.

4.4 Dataset Splitting

The notebook uses stratified splitting to maintain class representation. The described workflow first assigns 70% of samples to training and 30% to a temporary partition; the temporary partition is split equally into validation and test sets, giving nominal proportions of 70%, 15%, and 15%. Exact counts should be taken from the executed notebook. A storm-wise split is recommended for a stronger assessment of generalization to unseen storms.

4.5 Proposed MLFF-CycloneNet

The MLFF-CycloneNet model is a custom convolutional classifier. It extracts features at multiple levels, fuses the representations, and applies channel and spatial attention before producing class scores.

4.5.1 Convolutional Feature Extraction

Convolution, batch normalization, and ReLU activation are combined in feature blocks. These operations learn local patterns and build increasingly abstract representations.

4.5.2 Multi-Level Feature Fusion

Feature maps from different network depths are combined. The intention is to preserve local details while including deeper contextual information that may help distinguish cyclone intensity patterns.

4.5.3 Channel and Spatial Attention

Channel attention reweights feature channels, while spatial attention reweights image locations. Their combined use allows the network to learn which feature information is useful for classification.

4.5.4 Classification Head

The final layer generates scores for the three target categories. The predicted category is selected from the model output; confidence values can be calculated from the output probabilities.

4.6 Cyclone Detection Model

A separate `MLFFCycloneDetector` produces a binary output: Cyclone or No Cyclone. In the notebook, positive patches are derived from cyclone-centred arrays and negative patches are

generated from a corner region. Since corner patches may still contain cyclone-related cloud structures, this strategy can create noisy labels. The detector's performance should therefore be interpreted cautiously until tested on independently collected, verified background images.

4.7 System Architecture

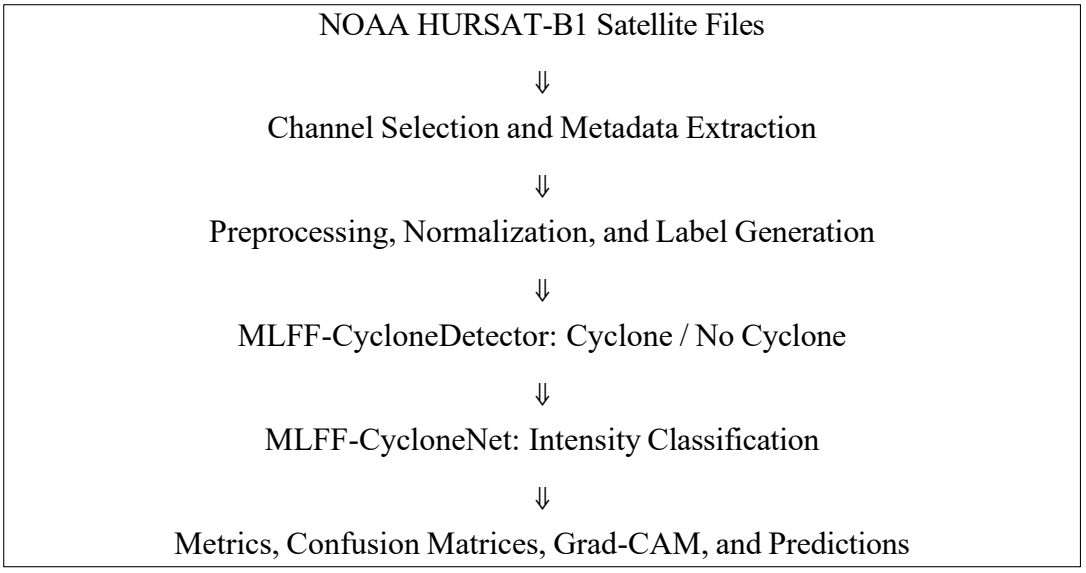


Figure 4.1: Overall architecture of the tropical cyclone analysis system

4.8 Loss Function

Cross-entropy loss is used for classification. For a one-hot target vector y and predicted class probabilities p, the loss is:

$$L_{CE} = -\sum_{c=1} y_c \log(p_c).$$

It penalizes predictions that assign low probability to the correct class.

4.9 Optimizer and Learning-Rate Strategy

The notebook configuration uses the Adam optimizer with an initial learning rate of 0.001 and weight decay of 10^{-4}. A ReduceLROnPlateau scheduler lowers the learning rate by a factor of 0.5 when validation loss fails to improve for the configured patience interval. Confirm these settings against the final notebook before submission.

Implementation and Experimental Setup

5.1 Implementation Environment

The project is implemented in a Google Colab notebook using Python. Colab provides notebook execution and may provide GPU acceleration. The code checks whether CUDA is available and selects the appropriate device.

5.2 Data Loading and Dataset Class

The data-loading pipeline uses xarray-compatible tools to read NetCDF variables. NumPy arrays are prepared for model input, and PyTorch Dataset and DataLoader components supply samples in batches. The same channel order and normalization must be maintained throughout training, validation, testing, and prediction.

5.3 Model Training Strategy

5.3.1 Training and Validation

The classifier is trained on the training partition. Validation loss and metrics are checked during training to monitor learning and help select the best checkpoint.

5.3.2 Batch Processing

DataLoader batches reduce memory requirements and support efficient training. The notebook's configured batch size is 16, subject to confirmation against the executed code.

5.3.3 Model Checkpointing

The model checkpoint associated with the best validation loss is saved so that evaluation can use the selected model rather than simply the final epoch.

5.3.4 Learning-Rate Scheduling

The scheduler reduces the learning rate when validation loss plateaus. This can support convergence, although the actual benefit must be judged from the training history.

5.4 Experimental Procedure

The experimental procedure is:

1. Load and inspect the satellite files.
2. Prepare the selected channels and metadata.
3. Generate intensity labels and prepare the input arrays.
4. Split the data into training, validation, and test sets.
5. Train MLFF-CycloneNet for intensity classification.
6. Train the binary cyclone detector separately.
7. Load the selected checkpoints and evaluate the held-out test data.
8. Generate confusion matrices, classification reports, training curves, and Grad-CAM visualizations.
9. Run sample predictions and record the actual output.

5.5 Prediction Pipeline

For a new processed sample, the pipeline applies the required channel order and normalization, passes the input through the detector and classifier, and returns the corresponding labels and probability scores. Predictions should be made only for inputs that follow the same format as the training data.

5.6 Saved Artifacts

The notebook may save trained model weights, training history, evaluation metrics, confusion matrices, plots, and prediction summaries. The final report should include the actual outputs generated by the completed run rather than estimated values.

Results and Performance Analysis

6.1 Overview

This chapter describes the evaluation procedure for the cyclone intensity classifier and the binary detector. All numeric values in the tables below must be replaced with the final values printed by the Colab notebook. No performance value should be assumed without experimental evidence.

6.2 Quantitative Performance Metrics

6.2.1 Accuracy

Accuracy is the fraction of all predictions that are correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

For multiclass evaluation, the metric is calculated from the total number of correct predictions and total samples.

6.2.2 Precision

Precision measures the proportion of predicted positives that are correct:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

6.2.3 Recall

Recall measures the proportion of actual positives correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

6.2.4 F1-Score

The F1-score is the harmonic mean of precision and recall:

$$F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

6.3 Intensity Classification Results

Table 6.1: MLFF-CycloneNet test performance

Metric	**Actual notebook result**
Accuracy	Enter actual value
Weighted Precision	Enter actual value
Weighted Recall	Enter actual value
Weighted F1-score	Enter actual value
Test sample count	Enter actual value

6.4 Binary Detection Results

Table 6.2: MLFFCycloneDetector test performance

Metric	**Actual notebook result**
Accuracy	Enter actual value
Precision	Enter actual value
Recall	Enter actual value
F1-score	Enter actual value
Test sample count	Enter actual value

6.5 Training and Validation Analysis

Training and validation curves show how model behaviour changes over epochs. A decrease in training loss indicates that the model is fitting the training data. If validation performance stops improving while training performance continues to improve, overfitting may be occurring. Insert the training and validation plots generated by the notebook.

6.6 Confusion Matrix Analysis

A confusion matrix provides class-wise information about correct predictions and misclassifications. For the intensity classifier, it can show whether Tropical Depression, Tropical Storm, and Hurricane/Typhoon observations are confused with one another. Insert the actual confusion matrix from the notebook and interpret the largest off-diagonal values.

6.7 Precision, Recall, and F1-Score Analysis

Class-wise metrics are important because the dataset may be imbalanced. A high overall accuracy can hide poor performance on a less frequent category. Macro and weighted averages should be reported when available, alongside per-class results.

6.8 Grad-CAM and Qualitative Analysis

Grad-CAM visualizations provide a qualitative view of regions associated with selected predictions. Compare correct and incorrect predictions to identify whether the model attends to meaningful cloud structures or irrelevant regions. Heatmaps should be treated as an aid to inspection rather than proof of model reliability.

6.9 Robustness and Generalization

A random sample-level split may allow observations from the same storm to occur in both training and test sets. This can lead to optimistic estimates. A storm-wise split, an independent test period, and verified non-cyclone samples would provide a more robust evaluation.

6.10 Summary of Findings

Complete this section after running the notebook. Summarize the final test metrics, class-wise strengths and weaknesses, detector results, and observations from the confusion matrix and Grad-CAM outputs. Clearly state that the current system classifies supplied satellite observations and does not yet provide validated future-track predictions.

Conclusion and Future Scope

7.1 Conclusion

This case study describes a deep-learning pipeline for identifying and classifying tropical cyclone patterns from multi-channel satellite observations. The proposed MLFF-CycloneNet architecture combines convolutional feature extraction, multi-level feature fusion, and channel and spatial attention. A separate model performs cyclone-versus-background classification. The implementation includes data preprocessing, training, checkpoint selection, evaluation metrics, confusion matrices, and Grad-CAM visualization. The usefulness of the system must be established through the actual test results and independent validation.

7.2 Limitations

- The performance values must be taken from the completed notebook run.
- Corner patches used as negative samples may contain cyclone-related features.
- Sample-level random splitting may not measure generalization to unseen storms.
- Data availability, missing channel values, class imbalance, and metadata quality may affect results.
- Grad-CAM is a qualitative explanation and cannot guarantee meteorological validity.
- The current implementation classifies supplied images and does not establish future cyclone-track forecasting.

7.3 Future Scope

7.3.1 Storm-Wise Evaluation

Partition the data by storm identity or time period so that test samples are independent of training observations.

7.3.2 Verified Background Dataset

Use independently collected and manually or scientifically verified non-cyclone imagery for detector training and testing.

7.3.3 Additional Satellite Channels

Evaluate additional channels and derived variables where data availability and physical meaning support their use.

7.3.4 Temporal Forecasting

Extend the image classifier with time-series models to study changes in cyclone intensity or movement across consecutive observations. Such a system requires temporally aligned data and separate forecasting evaluation.

7.3.5 Uncertainty Estimation

Estimate prediction uncertainty and flag low-confidence cases for expert review.

7.3.6 Operational Visualization

Develop a dashboard to display satellite inputs, predicted classes, confidence values, and explanatory heatmaps while clearly communicating model limitations.

Appendix I: Model Details and Preprocessing

8.1 MLFF-CycloneNet Model Details

The classifier accepts a three-channel satellite array. Convolutional blocks extract features at multiple depths. Multi-level feature fusion combines these representations, and channel and spatial attention refine the fused feature maps. A final classification head returns scores for the three intensity classes. The exact layer dimensions and parameter count should be copied from the model summary printed by the notebook.

8.2 Model Training Strategy

The notebook trains the classifier using cross-entropy loss and Adam optimization. Validation loss is monitored during training, and the checkpoint with the best validation performance is retained. The detector is trained as a separate binary classification model. Report the exact number of epochs completed, learning-rate settings, and batch size from the executed notebook.

8.3 Preprocessing Pipeline

1. Read the source NetCDF file.
2. Inspect available channel and metadata variables.
3. Extract IRWIN, IRWVP, and VSCHN where available.
4. Check array dimensions, missing values, and data type.
5. Combine channels into the expected model input shape.
6. Apply the same normalization used during training.
7. Associate the sample with its metadata-derived class label.
8. Convert the processed array into the tensor format expected by the model.

8.4 Data Augmentation

Only describe augmentation techniques that are actually implemented in the notebook. If no augmentation is used, state that the current experiment relies on preprocessing and normalization, and consider physically appropriate augmentation in future work.

8.5 Classification Head Details

The classification head converts the extracted feature representation into scores for three classes. A softmax operation can convert these scores into probabilities:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)},$$

where z_k is the score for class k. The class with the largest probability is selected as the prediction.

8.6 Important Hyperparameters

Table 8.1: Hyperparameters to verify against the executed notebook

Hyperparameter	**Configured value**
Optimizer	Adam
Initial learning rate	0.001 (verify notebook)
Weight decay	10^{-4} (verify notebook)
Batch size	16 (verify notebook)
Learning-rate scheduler	ReduceLROnPlateau (verify notebook)
Number of epochs	Insert actual completed epochs
Input channels	3 selected satellite channels
Number of intensity classes	3

References

9.1 Reference Categories

The sources are grouped into dataset documentation, deep-learning theory, attention and interpretability, optimization, and software documentation.

9.2 Bibliographic References

The bibliography below includes foundational research relevant to convolutional neural networks, attention, Grad-CAM, and Adam optimization, along with the dataset documentation.

9.3 Software and Online Resources

The implementation relies on Python scientific-computing libraries, PyTorch, xarray, and scikit-learn. Their documentation is included in the bibliography.

9.4 Reference Summary

The references support the theoretical background and implementation choices discussed in this report. NOAA documentation provides the dataset context, while research on CNNs, attention, Grad-CAM, and Adam provides methodological background. Software documentation supports the data-processing and model-development workflow.

[1] National Oceanic and Atmospheric Administration, *HURSAT-B1: Hurricane Satellite Data*, NOAA National Centers for Environmental Information, Dataset information. [Online]. Available: https://www.ncei.noaa.gov/products/hurricane-satellite-data

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791

[3] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "Cbam: Convolutional block attention module," in *Proceedings of the European Conference on Computer Vision*, 2018, pp. 3–19. [Online]. Available: https://arxiv.org/abs/1807.06521

[4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 618–626. [Online]. Available: https://arxiv.org/abs/1610.02391

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[6] PyTorch Contributors, *PyTorch Documentation*, Online documentation. [Online]. Available: https://pytorch.org/docs/stable/index.html

[7] xarray Developers, *xarray: N-D Labeled Arrays and Datasets in Python*, Online documentation. [Online]. Available: https://docs.xarray.dev/

[8] scikit-learn Developers, *scikit-learn: Machine Learning in Python*, Online documentation. [Online]. Available: https://scikit-learn.org/stable/

[9] NumPy Developers, *NumPy Documentation*, Online documentation. [Online]. Available: https://numpy.org/doc/

[10] Matplotlib Development Team, *Matplotlib Documentation*, Online documentation. [Online]. Available: https://matplotlib.org/stable/

[11] Unidata, *netCDF4-Python Documentation*, Online documentation. [Online]. Available: https://unidata.github.io/netcdf4-python/